



POLITECNICO
MILANO 1863



ITECNICO
LANO 1863

architettura dei calcolatori e sistemi operativi

RISC V – il processore capitolo 4 P&H

sommario

Instruction Set Architecture (ISA) di riferimento per
il processore
esecuzione delle istruzioni
struttura del processore monociclo



caratteristiche principali dell'architettura RISC V

architettura RISC (Reduced Instruction Set Computer):

esegue soltanto istruzioni semplici in un ciclo base ridotto e ha un numero ridotto di formati istruzione diversi

micro-architettura LOAD / STORE di tipo Harvard: gli operandi dell'ALU possono provenire soltanto dai registri di uso generale presenti nella CPU e **non** possono provenire direttamente dalla memoria

sono necessarie apposite istruzioni di

- *caricamento (load)* dei dati da memoria ai registri;
- *memorizzazione (store)* dei dati dai registri alla memoria.
- La memoria vista dal processore è suddivisa in *memoria istruzioni* e *memoria dati*, - sono «*memorie cache*» che contengono copie delle informazioni in memoria centrale per sfruttare la località spaziale e temporale dei programmi
 - *la coerenza dei loro contenuti con quelli della RAM è garantita dai meccanismi di gestione della memoria... Vedremo meglio in seguito*

micro-architettura pipeline: tecnica per migliorare le prestazioni basata sulla sovrapposizione temporale dell'esecuzione di più istruzioni appartenenti ad un flusso di esecuzione sequenziale



processore e set (insieme o repertorio) di istruzioni di riferimento

analizzeremo *un'implementazione semplificata* del processore **RISC V**, a cui è associato il corrispondente set istruzioni

l'**insieme ridotto di istruzioni** (istruzioni di riferimento per la realizzazione del processore) contiene le seguenti categorie

- istruzioni aritmetico-logiche: **add, sub, and, or** (formato R)
- istruzioni di trasferimento da / verso la memoria:
 - **ld** (formato I)
 - **sd** (formato S)
- istruzioni di salto condizionato **beq** (formato B, **SB** in PH)

nell'implementazione semplificata non vengono considerate le istruzioni nei formati U e J (**UJ** in PH)

possibile (e semplice) l'estensione ad altre istruzioni e/o altri formati



formato istruzioni completo – i

- *opcode* (codice operativo): 7 bit
 - ✓ opcode identico per «gruppi» di istruzioni: ad es. branch condizionale, load, store, aritmetiche / logiche immediate
- numero di registro (*rs1*, *rs2*, *rd*): 5 bit
 - ✓ la realizzazione dell'architettura prevede che la posizione di questi campi, nei diversi formati, sia sempre la stessa: questo ne semplifica la realizzazione
- *funct3* e *funct7* sono di fatto estensioni del codice operativo per gestire anche le operazioni dell'ALU
- *immediato* (12 o 20 bit) valore numerico in complemento a 2

formato degli *immediati* nelle istruzioni

	formato	immediato	estensione a 32 e 64 bit
aritmetiche/logiche	R	na	na
aritmetiche/logiche immediate	I	12 bit in cpl2	estensione del segno
load	I	12 bit in cpl2	estensione del segno
store	S	12 bit in cpl2	estensione del segno
branch condizionale	B (SB)	12 bit in cpl2, multiplo di 2	estensione del segno
lui	U	20 bit in cpl2	na
jump&link (NB: la jalr è in formato I)	J (UJ)	20 bit in cpl2, multiplo di 2	estensione del segno



formato istruzioni completo – ii

posizione dei campi nell'istruzione

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0				
funct7				rs2			rs1		funct3		rd			opcode		R-type		
imm[11:0]						rs1		funct3		rd			opcode		I-type			
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type		
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]		imm[11]		opcode		B-type
imm[31:12]										rd			opcode			U-type		
imm[20]		imm[10:1]				imm[11]		imm[19:12]				rd			opcode		J-type	

nb.
per S-type e B-type
stessa suddivisione,
cambi a valore di 1 bit
di immediato

nb.
per B-type e J-type
imm_0 == 0

costruzione degli immediati a 32 bit (64 con ulteriore estensione segno) a partire dai bit dell'istruzione, effettuata HW dal processore

31	30	20	19	12	11	10	5	4	1	0	
— inst[31] —						inst[30:25]	inst[24:21]		inst[20]		I-immediate
— inst[31] —						inst[30:25]	inst[11:8]		inst[7]		S-immediate
— inst[31] —					inst[7]	inst[30:25]	inst[11:8]		0		B-immediate
inst[31]	inst[30:20]			inst[19:12]		— 0 —					U-immediate
— inst[31] —				inst[19:12]		inst[20]	inst[30:25]	inst[24:21]		0	J-immediate



formato istruzioni dell'architettura di riferimento

Nome (posizione dei bit)	Campi					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) Tipo R	funz7	rs2	rs1	funz3	rd	codop
(b) Tipo I	immediate[11:0]		rs1	funz3	rd	codop
(c) Tipo S	immed[11:5]	rs2	rs1	funz3	immed[4:0]	codop
(d) Tipo SB	immed[12,10:5]	rs2	rs1	funz3	immed[4:1,11]	codop

opcode

add, sub, and, or	formato R	0110011	51 dec
ld	formato I	0000011	3 dec
sd	formato S	0100011	35 dec
beq	formato B	1100011	99 dec



esecuzione delle istruzioni



passi svolti durante l'esecuzione delle istruzioni aritmetico-logiche *di tipo R*

Istruzioni aritmetico-logiche: **op rd, rs1, rs2**

Prelievo Istruz. & Increm. PC	Lettura Registri Sorgente rs1 e rs2	Op. ALU sui Dati Letti (rs1 op rs2)	Scrittura nel Reg. Destinazione rd
-------------------------------	-------------------------------------	-------------------------------------	------------------------------------

un'istruzione aritmetico-logica (**tipo R**), e.g., **add rd, rs1, rs2**, viene eseguita in **4** passi

- prelievo istruzione dalla *memoria istruzioni* e incremento del *PC*
- lettura dei 2 registri sorgente (**rs1** e **rs2**) dal *banco dei registri*, utilizzando i bit [19-15] e [24 -20] per selezionare i registri
- operazione della ALU sui dati letti dal *banco dei registri*, utilizzando il campo **funct3 e funct7** per realizzare la funzione aritmetico-logica.
- scrittura del risultato dell'ALU nel *banco dei registri* utilizzando i bit [11-7] dell'istruzione per selezionare il registro destinazione (**rd**)

Nome (posizione dei bit)	Campi					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) Tipo R	funz7	rs2	rs1	funz3	rd	codop
(b) Tipo I	immediate[11:0]		rs1	funz3	rd	codop
(c) Tipo S	immed[11:5]	rs2	rs1	funz3	immed[4:0]	codop
(d) Tipo SB	immed[12,10:5]	rs2	rs1	funz3	immed[4:1,11]	codop



passi svolti durante l'esecuzione delle istruzioni di *load*

Istruzioni di *load*: **ld rd, offset12(rs1)**

Prelievo Istruz. & Increm. PC	Lettura Registro Base rs1	Op. ALU (rs1+offset_ext)	Prelievo Dato M(rs1+offset_ext)	Scrittura nel Reg. Destinazione rd
-------------------------------	---------------------------	--------------------------	---------------------------------	------------------------------------

un'istruzione di load (tipo I), e.g., **ld rd, offset12(rs1)**, viene eseguita in **5** passi:

- prelievo istruzione dalla *memoria istruzioni* e incremento del PC
- lettura del registro base (*rs1*) dal *banco dei registri*, bit [19-15]
- operazione della ALU per calcolare la somma del valore letto dal registro base e dell'immediato a 64 bit (*offset_ext*) costruito opportunamente dai 12 bit di *offset*
- prelievo dalla memoria dati usando come indirizzo di lettura il risultato della ALU
- scrittura del dato proveniente dalla *memoria dati* nel *banco dei registri*; il registro destinazione (*rd*) è indicato dai bit [11-7] dell'istruzione

Nome (posizione dei bit)	Campi					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) Tipo R	funz7	rs2	rs1	funz3	rd	codop
(b) Tipo I	immediate[11:0]		rs1	funz3	rd	codop
(c) Tipo S	immed[11:5]	rs2	rs1	funz3	immed[4:0]	codop
(d) Tipo SB	immed[12,10:5]	rs2	rs1	funz3	immed[4:1,11]	codop



passi svolti durante l'esecuzione delle istruzioni di store

Istruzioni di *store*: **sd rs2, offset12(rs1)**

Prelievo Istruz. & Increm. PC	Lettura Registri Base rs1 & Sorg. rs2	Op. ALU (rs1+offset_ext)	Scrittura Dato M(rs1+offset_ext)
----------------------------------	--	-----------------------------	-------------------------------------

un'istruzione di store (tipo S), e.g., **sd rs2, offset12(rs1)**, viene eseguita in **4** passi

- prelievo istruzione dalla *memoria istruzioni* e incremento del PC
- lettura del registro base (rs1), bit [19-15], e del registro sorgente (rs2), bit [24-20], dal *banco dei registri*
- operazione della ALU per calcolare la somma del valore letto dal registro base e dell'immediato a 64 bit (offset_ext) costruito opportunamente dai 12 bit di offset
- scrittura del dato proveniente dal registro sorgente (rs2) nella *memoria dati* utilizzando come indirizzo di scrittura il risultato della ALU

Nome (posizione dei bit)	Campi					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) Tipo R	funz7	rs2	rs1	funz3	rd	codop
(b) Tipo I	immediate[11:0]		rs1	funz3	rd	codop
(c) Tipo S	immed[11:5]	rs2	rs1	funz3	immed[4:0]	codop
(d) Tipo SB	immed[12,10:5]	rs2	rs1	funz3	immed[4:1,11]	codop



passi svolti durante l'esecuzione delle istruzioni di salto condizionato

Istruzioni di salto condizionato: **beq rs1, rs2, offset12**

Prelievo Istruz. & Increm. PC	Lettura Registri Sorgente rs1 e rs2	Op. ALU (rs1-rs2) & (PC+offset_ext)	Scrittura nel PC
-------------------------------	-------------------------------------	-------------------------------------	------------------

un'istruzione di salto condizionato (tipo B), e.g., **beq rs1, rs2, offset12**, viene eseguita in 4 passi

- prelievo istruzione dalla *memoria istruzioni* e incremento del PC
- lettura dei 2 registri sorgente (*rs1* e *rs2*) dal *banco dei registri*, bit [19-15] e [24 -20]
- operazione della ALU per effettuare la sottrazione tra i valori letti dal *banco dei registri*; il valore del PC viene sommato all'immediato a 64 bit (*offset_ext*) costruito opportunamente dai 12 bit di *offset*; poiché l'*offset* a 12 bit indica una distanza di salto misurata in numero di *half-word*, l'*offset_ext* a 64 bit deve essere fatto scorrere a sx di 1 posizione per riportarlo al byte; il risultato è l'indirizzo di destinazione del salto (Branch Target Address)
- l'uscita Zero dell'ALU viene utilizzata per decidere quale valore debba essere memorizzato nel PC: (PC + 4) oppure (PC + *offset_ext*)



passi svolti durante l'esecuzione delle istruzioni

Istruzioni aritmetico-logiche: **op rd,rs1,rs2**

Prelievo Istruz. & Increm. PC	Lettura Registri Sorgente rs1 e rs2	Op. ALU sui Dati Letti (rs1 op rs2)	Scrittura nel Reg. Destinazione rd
-------------------------------	-------------------------------------	-------------------------------------	------------------------------------

nb.

- I primi due passi sono comuni a tutte le classi di istruzioni
- Tutte le classi usano la ALU nel terzo passo, anche se ciascuna in modo diverso

Istruzioni di caricamento (*load*): **ld rd,offset12(rs1)**

Prelievo Istruz. & Increm. PC	Lettura Registro Base rs1	Op. ALU (rs1+offset_ext)	Prelievo Dato M(rs1+offset_ext)	Scrittura nel Reg. Destinazione rd
-------------------------------	---------------------------	--------------------------	---------------------------------	------------------------------------

Istruzioni di memorizzazione (*store*): **sd rs2,offset12(rs1)**

Prelievo Istruz. & Increm. PC	Lettura Registri Base rs1 & rs2	Op. ALU (rs1+offset_ext)	Scrittura Dato M(rs1+offset_ext)
-------------------------------	---------------------------------	--------------------------	----------------------------------

Istruzioni di salto condizionato: **beq rs1,rs2,offset12**

Prelievo Istruz. & Increm. PC	Lettura Registri Sorgente rs1 e rs2	Op. ALU (rs1-rs2) & (PC+offset_ext)	Scrittura nel PC
-------------------------------	-------------------------------------	-------------------------------------	------------------



esecuzione delle istruzioni – i

per ogni **tipo di istruzione** i primi due passi da eseguire sono identici

- inviare il contenuto del *Program Counter (PC)* a una memoria che contiene il codice per prelevare l'istruzione (*Memoria Istruzioni*)
- leggere uno o due registri dal banco dei registri utilizzando i campi dell'istruzione per selezionare i registri ai quali accedere

dopo questi due passi, le azioni necessarie per concludere l'esecuzione dell'istruzione dipendono dal **tipo di istruzione** (codice operativo), sebbene tutte le istruzioni utilizzino l'*ALU* dopo la lettura dei registri

- le istruzioni aritmetico-logiche per l'esecuzione dell'operazione
- le istruzioni di riferimento a memoria per il calcolo dell'indirizzo effettivo
- i salti condizionati per valutare l'esito dei confronti



esecuzione delle istruzioni – ii

dopo avere utilizzata la ALU, le azioni richieste per completare le varie istruzioni si differenziano ulteriormente

- le istruzioni aritmetico-logiche devono scrivere nel registro destinazione il risultato dell'ALU
- le istruzioni di *load* richiedono l'accesso in lettura alla *Memoria Dati* ed eseguono il caricamento del dato letto nel registro destinazione
- le istruzioni di *store* richiedono l'accesso in scrittura alla *Memoria Dati* ed eseguono la memorizzazione del dato proveniente dal registro sorgente
- le istruzioni di salto condizionato possono cambiare l'indirizzo dell'istruzione successiva in base al risultato del confronto



unità funzionali richieste durante l'esecuzione delle istruzioni di riferimento

Tipo di istruzione	Unità funzionali utilizzate			
Tipo R	Memoria istruzioni	Banco registri	ALU	Banco registri
Caricamento parola	Memoria istruzioni	Banco registri	ALU	Memoria dati Banco registri
Memorizzazione parola	Memoria istruzioni	Banco registri	ALU	Memoria dati
Salto condizionato	Memoria istruzioni	Banco registri	ALU	

Nessun conflitto di Unità Funzionali per ogni istruzione
Banco registri usato PRIMA in lettura, POI in scrittura



tempo di esecuzione delle istruzioni

Tipo di istruzione	Lettura dell'istruzione	Lettura dei registri	Operazione con la ALU	Accesso ai dati in memoria	Scrittura del register file	Tempo totale
Load doubleword (ld)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store doubleword (sd)	200 ps	100 ps	200 ps	200 ps		700 ps
Formato R (add, sub e or)	200 ps	100 ps	200 ps		100 ps	600 ps
Salto condizionato (beq)	200 ps	100 ps	200 ps			500 ps

PC non contiene ritardo perché la maggior parte è distribuito sulla porta di scrittura
NON PRESENTE nel ingolo registro PC

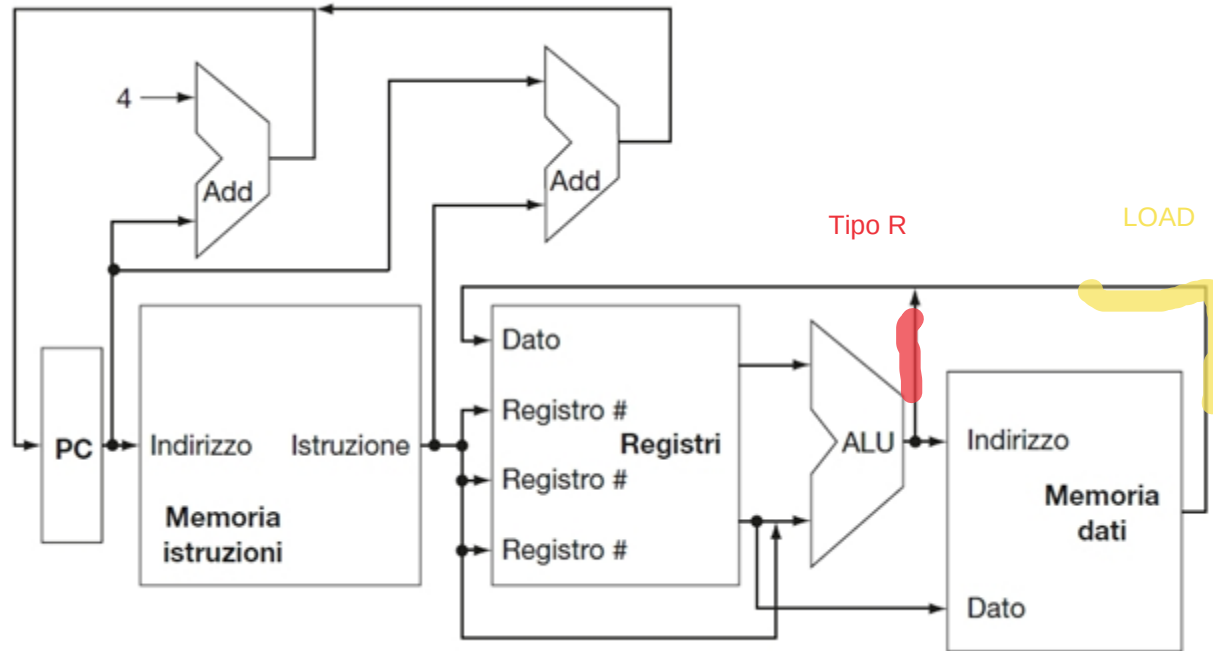
- nota: se il processore deve effettuare ogni istruzione in un solo ciclo di clock, la durata del ciclo non deve essere inferiore a *800 ps*, perché si deve tenere conto dell'istruzione più lenta



(Micro-)architettura del processore



struttura base del processore RISC V – i



- **Memoria Istruzioni (MI)** (usata solo in lettura) separata dalla **Memoria Dati (MD)**
 - allineamento istruzioni e dati in memoria: un solo accesso a memoria
- i 32 registri del processore sono organizzati in un **Register File - RF** con due porte di lettura e una porta in scrittura.
 - i campi dell'istruzione indicano direttamente i registri che debbono essere utilizzati come operandi dell'istruzione e vengono perciò collegati agli ingressi del Register File

struttura base del processore RISC V – ii

il *Register File* ha 4 ingressi e 2 uscite, che realizzano due porte di lettura e una porta di scrittura

- 3 ingressi sono collegati ai campi dell'istruzione che specificano i registri sorgente o base e il registro destinazione (2 per porte in lettura e 1 per porta in scrittura)
- 1 ingresso è per i dati che possono essere scritti nel registro destinazione (per la porta di scrittura)
- 2 uscite del banco dei registri riportano il contenuto dei 2 registri letti (per le porte di lettura)

gli operandi della **ALU** sono utilizzati per

- calcolare un risultato aritmetico (per un'istruzione aritmetico-logica)
- calcolare un indirizzo di memoria (per load/store)
- effettuare un confronto (per un salto condizionato)

il risultato della ALU è utilizzato per

- scrittura in un registro destinazione (se l'istruzione è aritmetico-logica)
- essere usato come indirizzo di lettura/scrittura della Memoria Dati (se l'istruzione è load/store)

Occorre anche calcolare l'indirizzo della prossima istruzione, secondo un'apposita logica di controllo, se l'istruzione è di salto condizionato



metodologia di temporizzazione degli elementi sequenziali in RISC V e convenzioni per i segnali di controllo

temporizzazione degli elementi di stato (elementi sequenziali) *memoria istruzioni, memoria dati e bando di registri*

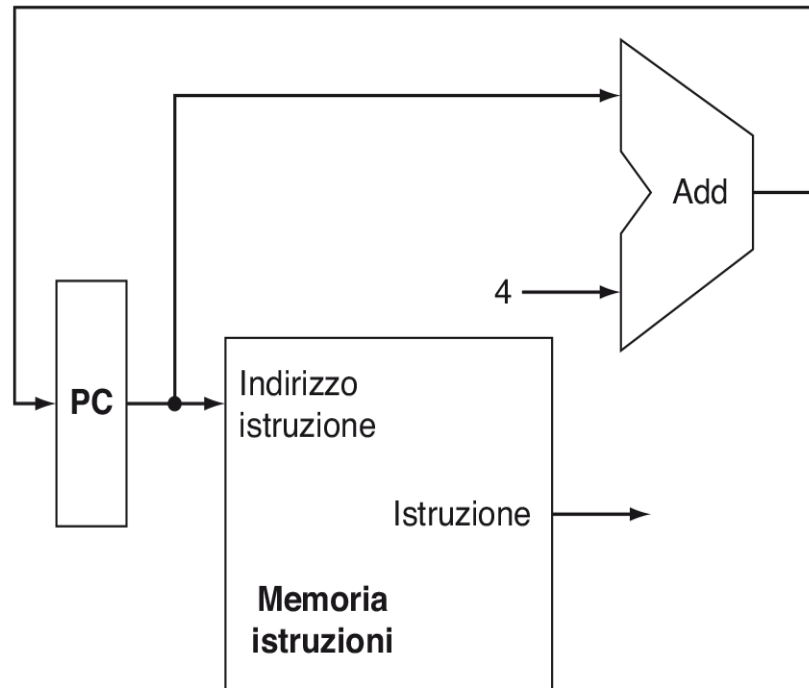
- sensibile ai fronti (*edge-triggered*): in corrispondenza del **fronte attivo del segnale di clock**
- **fronte attivo: di salita**
- **operazione efficace solo se** in corrispondenza del **segnale di controllo** dell'operazione asserito **e** del **fronte attivo del segnale di clock**

convenzione: non si riporta esplicitamente il segnale di controllo dell'operazione se questo risulta asserito *a ogni fronte attivo* del segnale di clock

- **Memoria Istruzioni:** viene solo letta a ogni ciclo di clock → non c'è segnale di controllo esplicito
- **registro PC:** viene letto e scritto a ogni ciclo di clock quindi non ci sono segnali di controllo espliciti
- **Register File:** i registri vengono letti a ogni ciclo e «scritti solo» in determinate istruzioni
 - ✓ segnale di controllo per la scrittura: **RegWrite**
- **Memoria Dati:** «lettura o scrittura solo» in determinate istruzioni
 - ✓ segnali di controllo separati per la lettura e scrittura **MemRead** e **MemWrite** (in questo modo è possibile avere la configurazione «memoria non acceduta»)

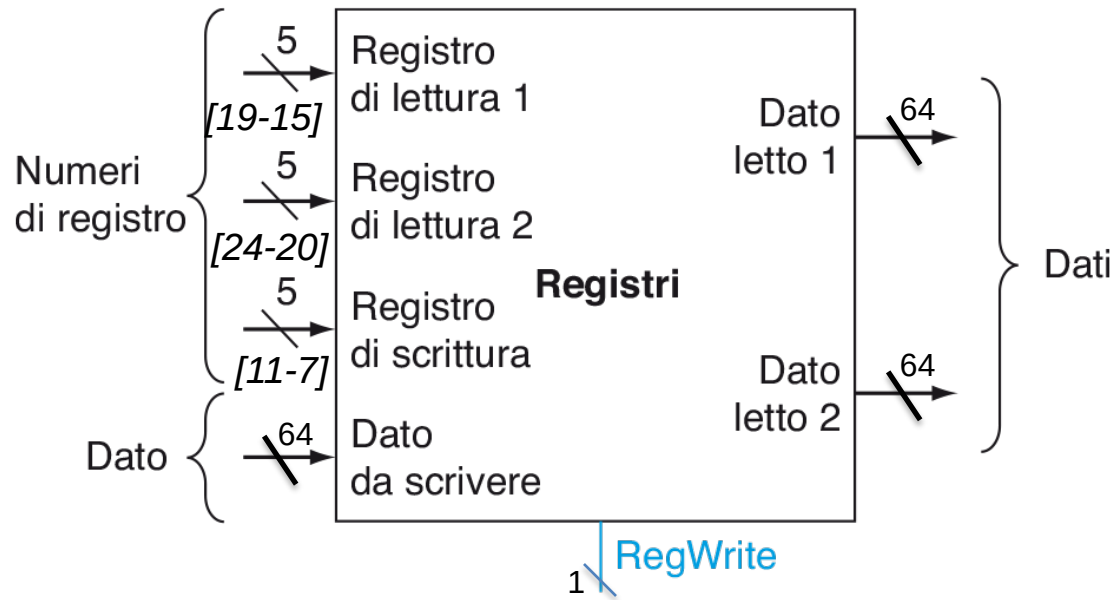


esecuzione della fase di caricamento (*fetch*) delle istruzioni

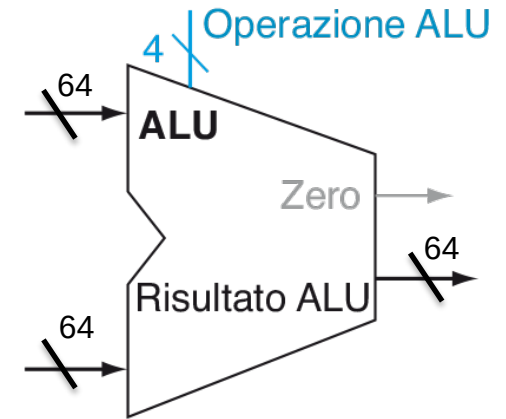


- *Memoria Istruzioni*
- *Program Counter (PC)* per memorizzare l'indirizzo dell'istruzione corrente;
- un *Sommatore (Add_addr)* per incrementare il *PC* in modo da poter indirizzare l'istruzione successiva ($PC + 4$).

esecuzione delle istruzioni aritmetico-logiche



a. Registri



b. ALU

- *Register File*
- *ALU* a 64 bit che riceve 2 ingressi da 64 bit e che restituisce un risultato da 64 bit

esecuzione delle istruzioni aritmetico-logiche (cont.)

le istruzioni aritmetico-logiche hanno come operandi **3 registri**: per ogni istruzione si devono leggere due parole di dati dal banco di registri e se ne deve scrivere una

per ogni parola letta dai registri sorgente sono necessari un ingresso (*5 bit*) al banco di registri, per specificare il numero del registro che si vuole leggere, ed un'uscita dal banco (*64 bit*) per il valore letto dal registro

per scrivere una parola nel registro destinazione sono necessari due ingressi: un ingresso (*5 bit*) per specificare il numero del registro in cui si vuole scrivere e un ingresso (*64 bit*) per il dato da scrivere

il banco di registri fornisce sempre in uscita il contenuto dei registri di lettura, mentre le scritture sono controllate da un apposito segnale di controllo della scrittura (*RegWrite*)

il segnale di controllo *OP_ALU* provvede a specificare all'ALU il tipo di operazione



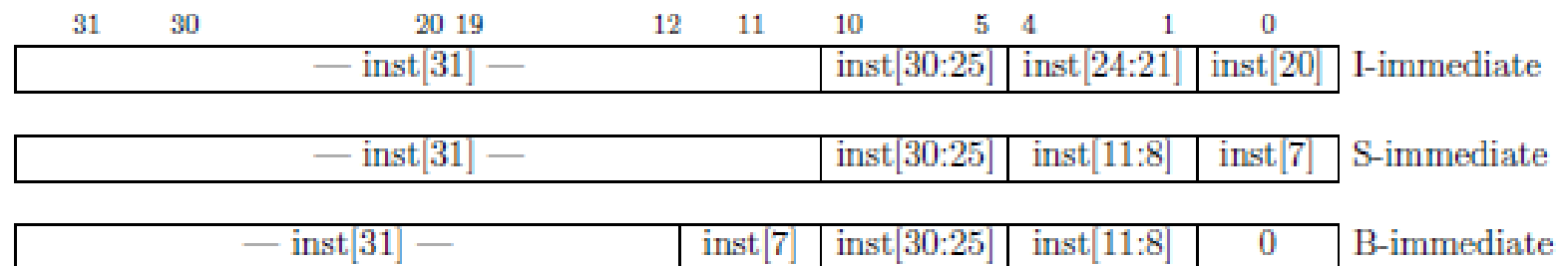
la costruzione degli immediati a seconda del formato istruzione – i

le istruzioni dell'architettura di riferimento hanno tutti immediati a 12 bit che rappresentano un *offset* da sommare al contenuto di un registro per ottenere un indirizzo di memoria:

- **ld** (formato I): *offset* è un valore a byte
- **sd** (formato S): *offset* è un valore a byte
- **beq** (formato B): *offset* è un valore a mezza parola e per essere sommato al PC per calcolare l'indirizzo di destinazione deve essere riportato a offset a byte

la costruzione dell'immediato a 64 bit viene realizzata da uno specifico modulo della micro-architettura: **Estensione Segno**

si noti che la modalità di costruzione è diversa per i 3 formati istruzione (I, S, B)



la costruzione degli immediati a seconda del formato istruzione – ii

Est. Segno riceve in ingresso i 32 bit dell'istruzione e ha quindi a disposizione i 7 bit del codice operativo opcode [6-0] per costruire l'estensione in modo corretto

- opcode[6] distingue tra ld/sd (0) e beq (1)
- opcode[5] distingue tra ld (0) e sd (1)
- l'estensione viene effettuata in parallelo nei 3 formati, concatenando opportunamente i bit di offset12 presenti nell'istruzione e estendendo il segno
- i due bit di codice operativo possono controllare un mux3:1 per selezionare l'estensione appropriata a seconda dell'istruzione in esecuzione

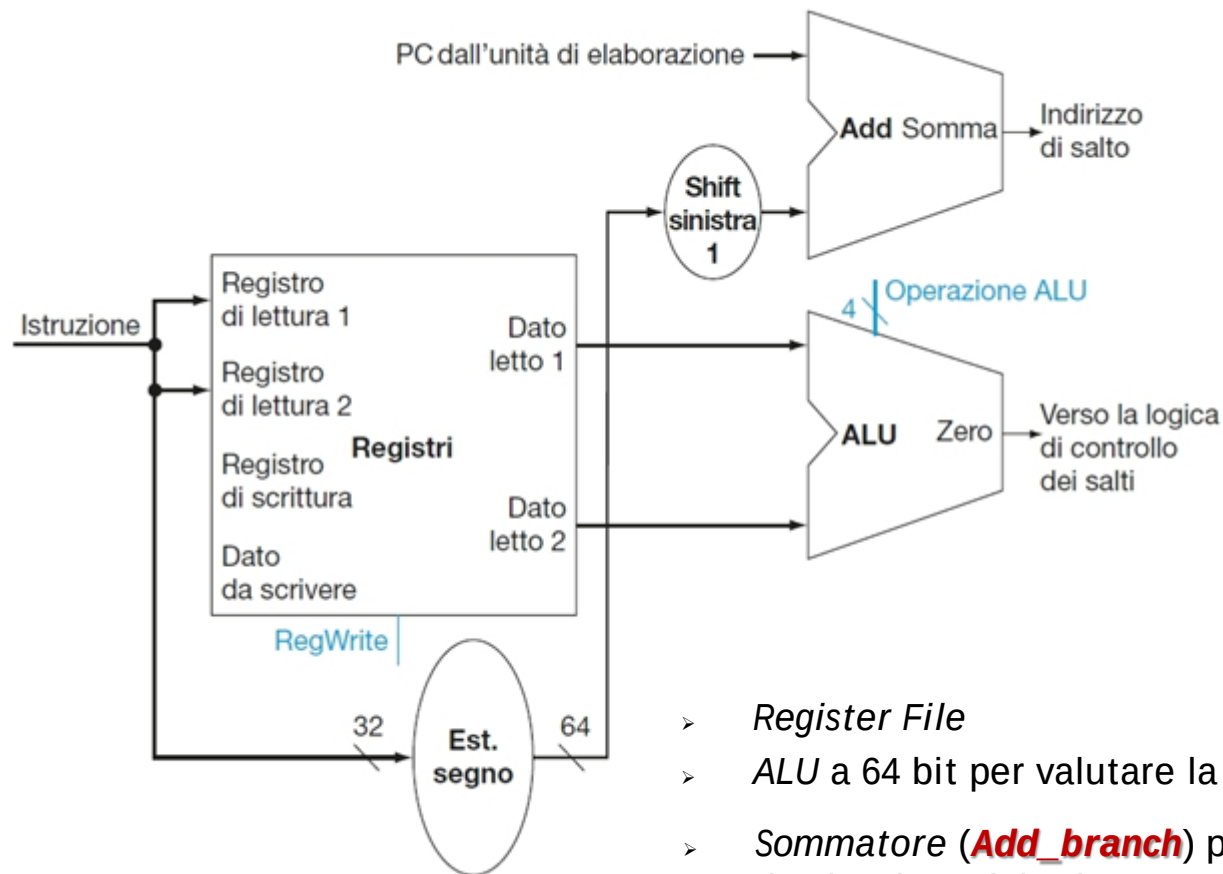
opcode			
add/sub/and/or	R	0110011	51 _{dec}
ld	I	0000011	3 _{dec}
sd	S	0100011	35 _{dec}
beq	B	1100011	99 _{dec}

l'estensione ottenuta è completa per ld/sd (l'offset è a byte) e può essere utilizzata per il calcolo dell'indirizzo di memoria

nel caso di branch beq (formato B, e di formato J che non trattiamo qui) i 64 bit, prima di essere sommati al valore del PC, vengono moltiplicati per 2 in modo da portare l'offset da una rappresentazione a mezza parola ad una rappresentazione a byte (shift a sx di 1)



esecuzione delle istruzioni di salto condizionato – i



- *Register File*
- *ALU a 64 bit per valutare la condizione di salto*
- *Sommatore (**Add_branch**) per calcolare l'indirizzo di destinazione del salto*
- *unità per l'**estensione del segno** e uno **shifter** a sinistra di 1 bit*
- *logica di controllo per decidere, in base al valore dell'uscita **Zero** dell'ALU, il nuovo valore del PC*



esecuzione delle istruzioni di salto condizionato – i

la base per il calcolo dell'indirizzo di destinazione di un salto condizionato è l'indirizzo dell'istruzione di salto (PC)

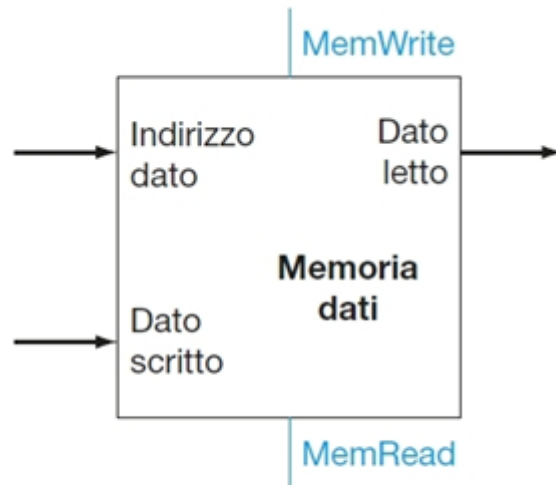
l'operazione di salto condizionato è costituita da due operazioni

- calcolo dell'indirizzo di destinazione del salto tramite un sommatore che effettua la somma tra il *PC* e il valore *dell'offset* esteso e fatto scorrere a sinistra di 1 bit
- confronto nell'ALU del contenuto dei registri operandi letti dal *RF*
 - se il segnale di uscita Zero dell'ALU è asserito $\Rightarrow$ la condizione di salto è verificata e l'indirizzo di destinazione del salto diventa il nuovo *PC*
 - se invece la condizione non è verificata $\Rightarrow$ il *PC* incrementato sostituisce il *PC* attuale
- per potere caricare nel *PC* l'indirizzo corretto della prossima istruzione da eseguire è necessario aggiungere un multiplexer a due vie (**MUX Branch**, negli schemi dell'architettura) in ingresso al *PC* stesso
- la selezione dell'indirizzo corretto è effettuata da un segnale di controllo che dipende dall'esito del confronto calcolato dalla ALU e dal fatto che si tratti di una istruzione di branch

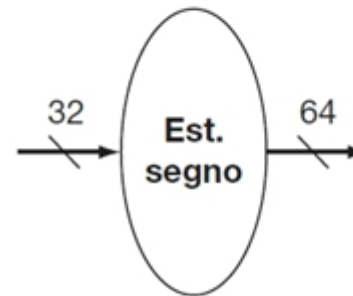


esecuzione delle istruzioni di *load* / *store* – i

- *Register File* per accesso al registro base dell'indirizzo di memoria e al registro destinazione (ld) o sorgente (sd)
- modulo estensione segno per generare lo spiazzamento su 64 bit
- ALU a 64 bit per calcolare l'indirizzo
- *Memoria Dati* da cui leggere (ld), *Memoria Dati* su cui scrivere (sd)



a. Unità di memoria dati



b. Unità di estensione del segno

esecuzione delle istruzioni di *load* / *store* – ii

- il valore dello spiazzamento (*offset*), dopo l'estensione di segno, è utilizzato come secondo operando della *ALU*
- ogni operazione è costituita da lettura del *RF*, calcolo nella *ALU* di un indirizzo di lettura/scrittura per accedere alla *Memoria Dati*, lettura/scrittura della *Memoria Dati* e, nel caso di *load*, scrittura del *RF*
- durante la *load*, il valore letto dalla *Memoria Dati* deve essere scritto nel registro destinazione del *RF*
- durante la *store*, il valore da scrivere nella *Memoria Dati* deve essere letto dal registro sorgente del *RF*



realizzazione del processore completo – i

esaminati gli elementi richiesti da ogni tipo di operazione, è possibile combinarli in un'unica unità di elaborazione

si assume che tutte le istruzioni siano eseguite in un solo ciclo di clock

- nessuna risorsa può essere utilizzata più di una volta per istruzione
- qualsiasi risorsa di cui si ha bisogno più di una volta deve essere **duplicata**

occorre quindi una *Memoria Istruzioni* **distinta** dalla *Memoria Dati* (memorie cache)



realizzazione del processore completo – ii

alcune unità funzionali possono essere **condivise** da **differenti flussi di istruzioni**

per condividere un componente hardware tra due diversi tipi di istruzione, si deve introdurre un **multiplexer** di dati per permettere connessioni multiple all'ingresso di un componente e selezionare uno tra i vari ingressi in base alla configurazione delle linee di controllo



esecuzione delle istruzioni aritmetico-logiche e *load / store* – i

da risolvere

1. il secondo ingresso della ALU è il contenuto di un registro (istruzione di tipo R) oppure è l'offset esteso (istruzione di *load / store*)

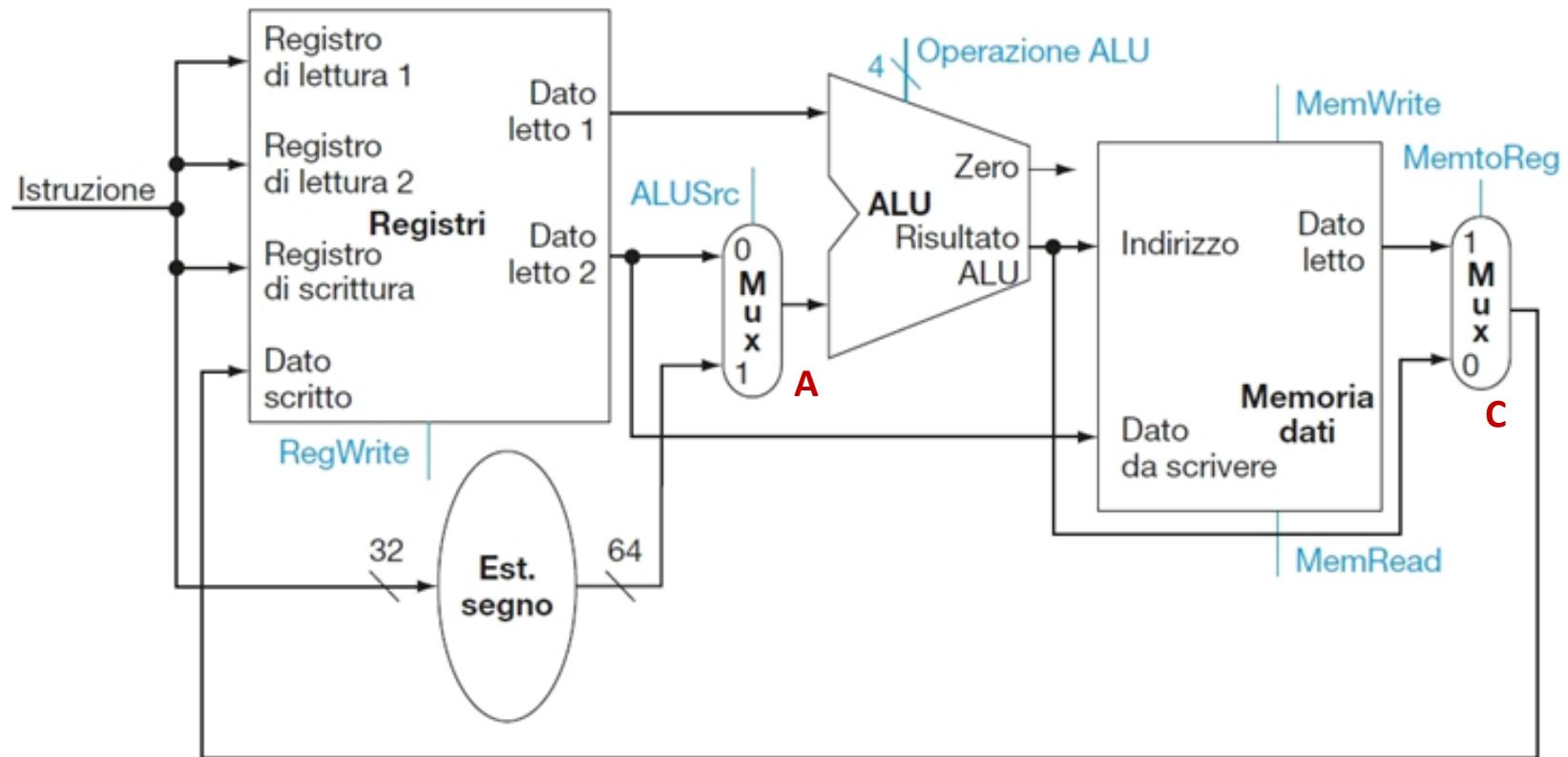
⇒ MUX al secondo ingresso della ALU (**Mux A**)

2. il valore scritto nel registro destinazione proviene dal risultato della ALU (istruzione tipo R) oppure dalla Memoria Dati (istruzione di *load*)

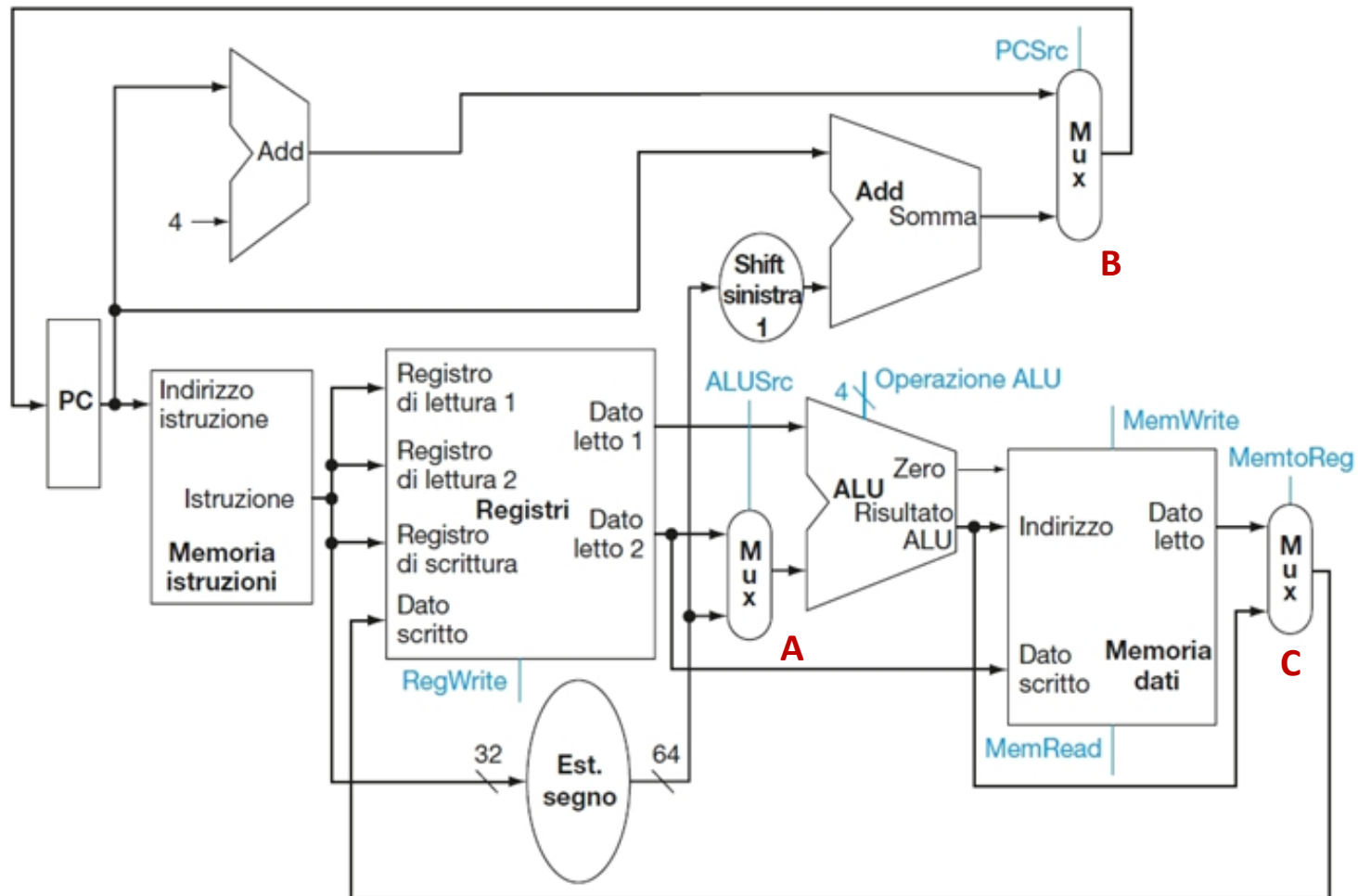
⇒ MUX all'ingresso dei dati per scrittura del RF (**Mux C**)



esecuzione delle istruzioni aritmetico-logiche e *load* / *store* – ii



struttura della CPU

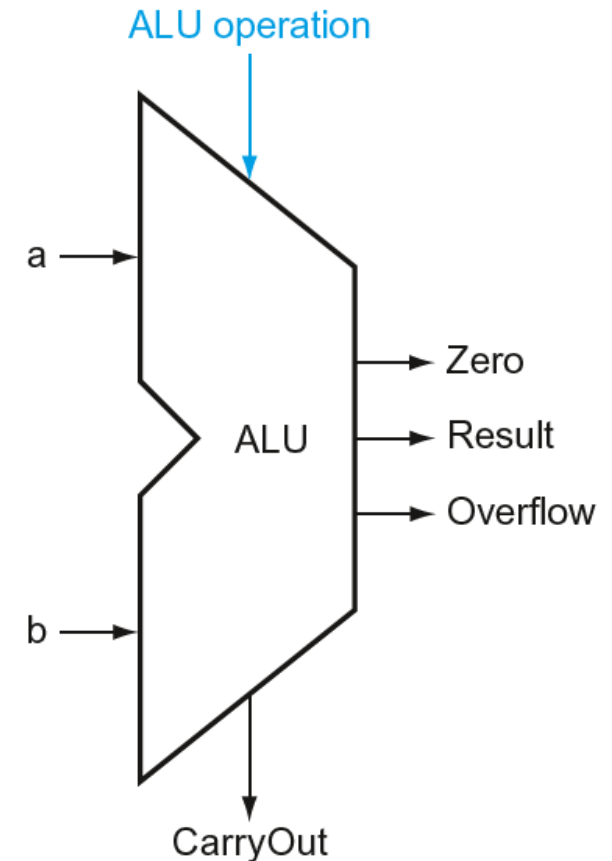


ALU di RISC V e linee di controllo

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set less than
1100	NOR

le linee di controllo interne della ALU sono:

Ainvert, Bnegate e Operation (2 bit)



... come blocco funzionale

il controllore della ALU

il controllore ALU genera i segnali di controllo alla ALU (**operazioni ALU**) in funzione degli ingressi

- **ALUOp**, determinati dai bit di codice operativo
- i campi **funz3** e **funz7**, che sono di fatto estensioni del codice operativo per finalizzare le operazioni dell'ALU e servono nelle istruzioni di tipo R

se ALUOp = 00 o 01 i campi funz7 e funz3 sono «indifferenti»

se ALUOp = 10 (istruzioni R) serve **istr 30** (di funz7) e **istr 12-14** (funz3)

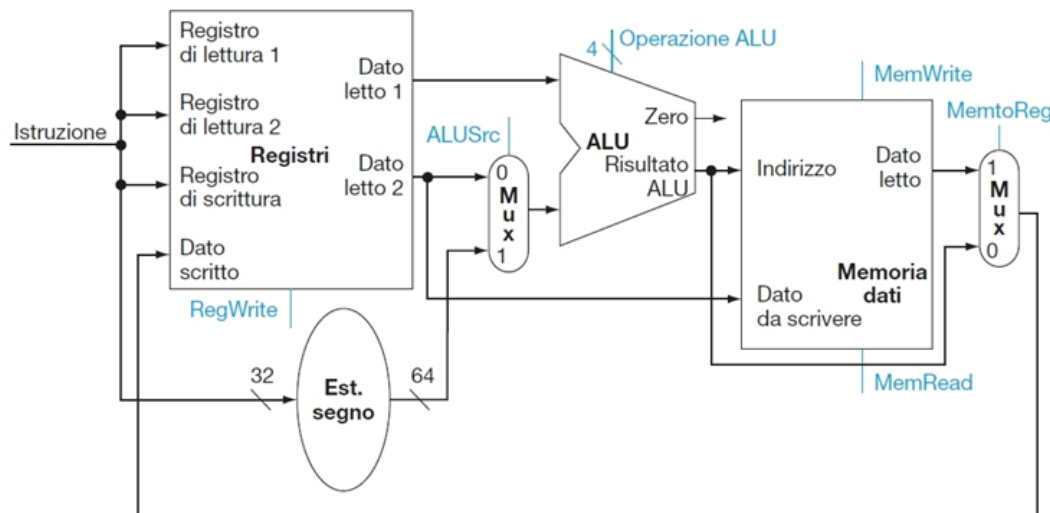
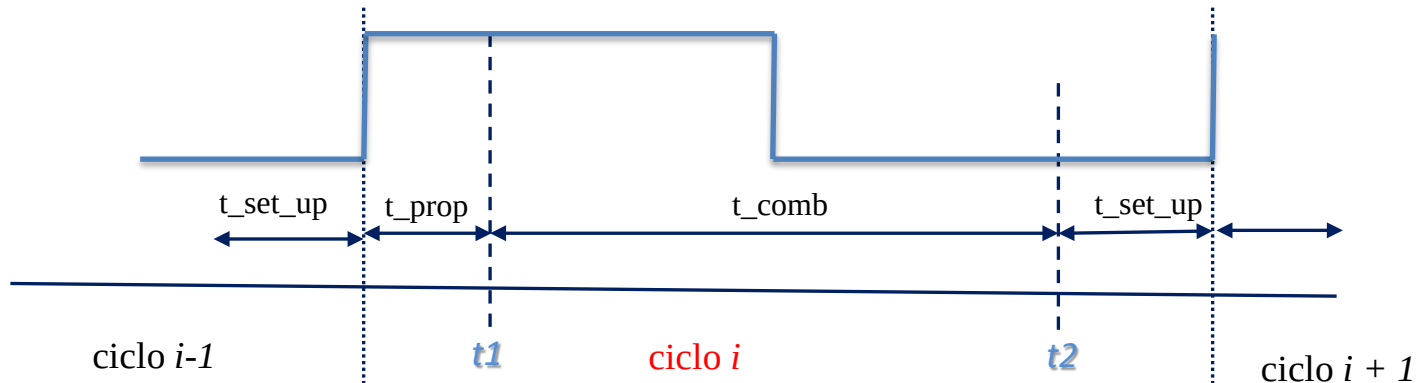
Codice operativo istruzione	ALUOp	Operazione eseguita dall'istruzione	Campo funz7	Campo funz3	Operazione dell'ALU	Ingresso di controllo alla ALU
ld	00	load di 1 parola doppia	XXXXXXXX	XXX	somma	0010
sd	00	store di 1 parola doppia	XXXXXXXX	XXX	somma	0010
beq	01	salto condizionato all'uguaglianza	XXXXXXXX	XXX	sottrazione	0110
Tipo R	10	add	0000000	000	somma	0010
Tipo R	10	sub	0100000	000	sottrazione	0110
Tipo R	10	and	0000000	111	AND	0000
Tipo R	10	or	0000000	110	OR	0001



nota sulla temporizzazione edge-triggered

nelle istruzioni aritmetiche (R) un registro sorgente può anche essere destinazione!!!!

$T_{clock} = T_{prop} + T_{set_up} + T_{comb}$ dove **T_{comb}** è sostanzialmente il tempo di operazione della ALU

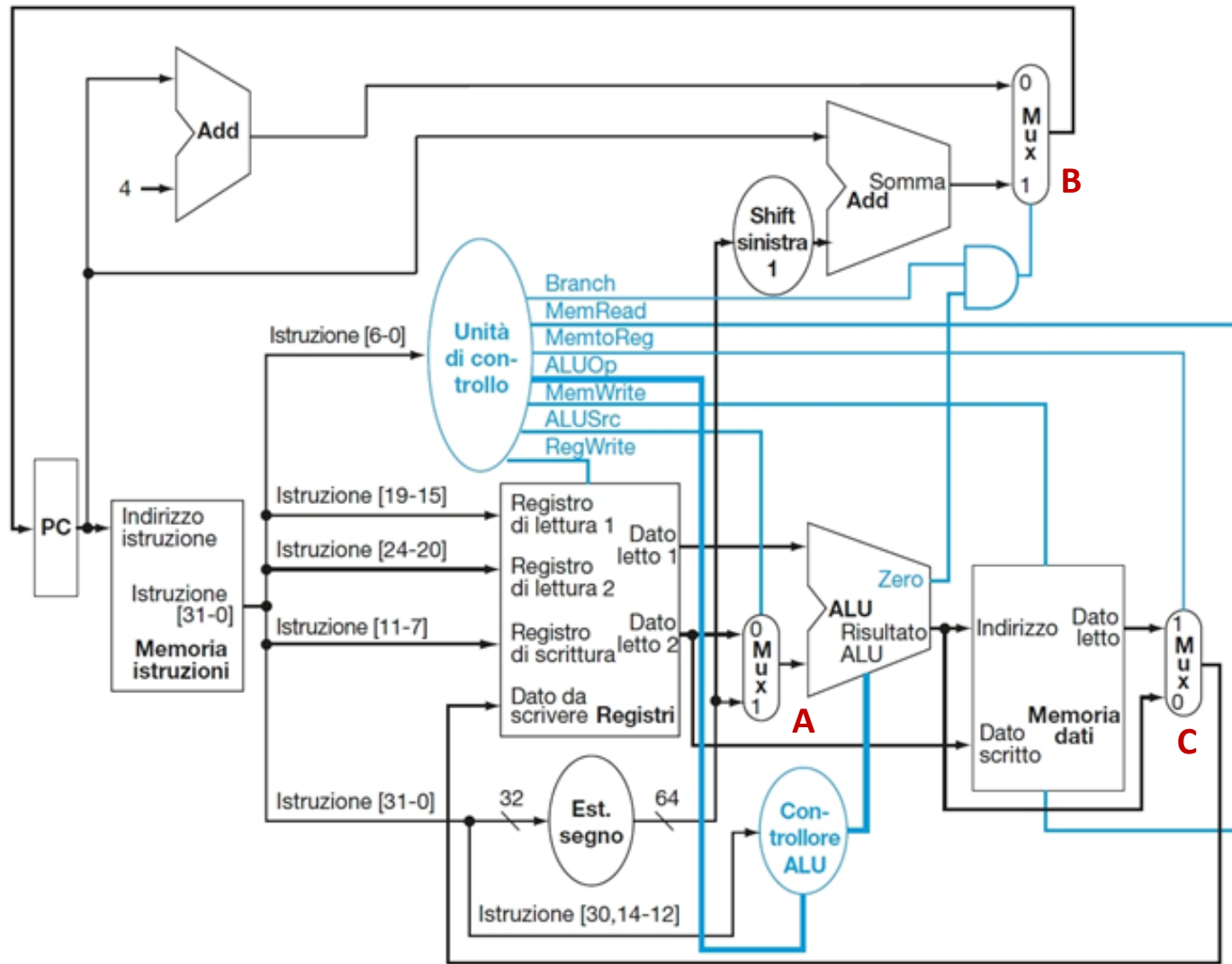


i segnali di controllo

Nome del segnale	Effetto quando non asserito	Effetto quando asserito
RegWrite	Nulla	Il dato viene scritto nel register file nel registro individuato dal numero del registro di scrittura
ALUSrc	Il secondo operando della ALU proviene dalla seconda uscita del register file (Dato letto 2)	Il secondo operando della ALU proviene dall'estensione del segno dei 12 bit del campo immediato dell'istruzione
PCSrc	Nel PC viene scritta l'uscita del sommatore che calcola il valore di $PC + 4$	Nel PC viene scritta l'uscita del sommatore che calcola l'indirizzo di salto
MemRead	Nulla	Il dato della memoria nella posizione puntata dall'indirizzo viene inviato in uscita sulla linea "Dato letto"
MemWrite	Nulla	Il contenuto della memoria nella posizione puntata dall'indirizzo viene sostituito con il dato presente sulla linea "Dato scritto"
MemtoReg	Il dato inviato al register file per la scrittura proviene dalla ALU	Il dato inviato al register file per la scrittura proviene dalla Memoria Dati



struttura della CPU con unità di controllo



codice operativo e segnali di controllo

Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
ld	1	1	1	1	0	0	0	0
sd	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1



funzione logica dell'unità di controllo

Input o output	Nome del segnale	Formato R	Id	sd	beq
Input	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Output	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp01	1	0	0	0
	ALUOp02	0	0	0	1

la tabella sopra rappresenta (in formato non usuale) la **tabella della verità** del circuito che implementa l'unità di controllo in cui sono rappresentate le sole configurazioni di interesse degli ingressi I[6]...I[0] (cod. Operativo) e i corrispondenti valori di uscita ALUSrc...ALUOp02



... semplice «esercizio» di estensione dell'architettura

..... e per eseguire anche la **bne**?

la logica di controllo per l'istruzione di salto condizionato

$$\text{PCsrc} = \text{Zero and Branch}$$



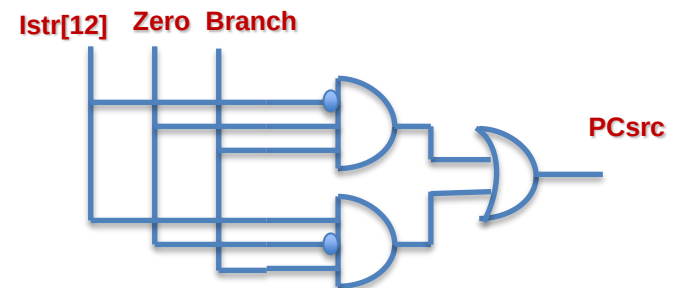
seleziona per la **beq** l'indirizzo della prossima istruzione da eseguire (PCsrc = 1 salta, PCsrc = 0, in sequenza)

- per **beq** si salta se il bit Zero = 1 e cioè se i due registri selezionati hanno valori identici
- per **bne** si salta se il bit Zero = 0 e cioè se i due registri selezionati **non** hanno valori identici

possiamo modificare la logica di controllo che genera il segnale **PCsrc** per distinguere **beq** da **bne**, tenendo conto dei bit di *funz3*, infatti

- **beq** – Op_code = **bne** – Op_code = 1100011 (99 dec)
- **beq** – funz3 = 000, **bne** – funz3 = 001
- quindi differiscono per il bit **istr[12]**

PCsrc =
(not Istr[12] and Zero and Branch)
or (Istr[12] and notZero and Branch)



implementazione della CPU a singolo ciclo

- si avvia all'esecuzione un'istruzione per ogni ciclo di clock e ogni istruzione deve essere iniziata e completata in un solo ciclo di clock
- il ciclo di clock deve avere la stessa durata per ogni istruzione

Tipo di istruzione	Lettura dell'istruzione	Lettura dei registri	Operazione con la ALU	Accesso ai dati in memoria	Scrittura del register file	Tempo totale
Load doubleword (ld)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store doubleword (sd)	200 ps	100 ps	200 ps	200 ps		700 ps
Formato R (add, sub e or)	200 ps	100 ps	200 ps		100 ps	600 ps
Salto condizionato (beq)	200 ps	100 ps	200 ps			500 ps

- la lunghezza del ciclo di clock è determinata dal percorso più lungo (percorso critico o *critical path*)
- l'implementazione su singolo ciclo non è molto veloce perché le altre istruzioni potrebbero essere implementate con un ciclo di clock più breve: nell'esempio le istruzioni tipo R richiedono $T = 600 \text{ ps}$, le istruzioni di store $T = 700 \text{ ps}$, istruzioni di salto condizionato $T = 500 \text{ ps}$ e istruzioni di salto incondizionato può essere considerato pari a 200 ps

